

مشکل (Problem)

یه کلاس کاربردی متدی که نیاز داری رو نداره و نمی‌تونی متد رو بهش اضافه کنی.

راه حل (Solution)

متد رو به کلاس کلاینت اضافه کن و یه شیء از کلاس کاربردی رو به عنوان آرگومان بهش پاس بده.

مثال از صفحه

قبل:

```
csharp

class Report
{
    ()void SendReport
    }

;DateTime nextDay = previousEnd.AddDays(1)
... //
{
    {
```

بعد:

```
csharp

class Report
{
    ()void SendReport
    }

;DateTime nextDay = NextDay(previousEnd)
... //
{

private static DateTime NextDay(DateTime date)
    }

;return date.AddDays(1)
```

چرا ریفکتور کنیم؟ (Why Refactor)

دلیل	توضیح
کتابخانه ثالث	کلاس توی یه کتابخانه ست که نمی‌تونی تغییرش بدی
کد تکراری	اگه این منطق چند جا تکرار شده، ارزشش رو داره که Foreign Method بسازی

مزایا (Benefits)

حذف کد تکراری. حتی اگه Foreign Method جای ایده‌آلی نداره، از تکرار کد بهتره.

نقطه ضعف (Drawbacks)

مشکل	راه‌حل
مشخص نبودن دلیل	با کامنت // Foreign Method توضیح بده
اگه چندتا Foreign Method داری	از Introduce Local Extension استفاده کن (Wrapper بساز)

How to Refactor (چطور انجام بدیم)

قدم	توضیح
۱	یه متد جدید در کلاس کلاینت بساز

توضیح	قدم
یه پارامتر برای شیء کلاس کاربردی بذار (اگه از خود کلاینت قابل گرفته، لازم نیست)	۲
قطعات کد مربوطه رو به این متد منتقل کن و با صدا زدن متد جایگزین کن	۳
با کامنت // Foreign Method مشخص کن که چرا این متد اینجاست	۴

تمرین

این کد رو با **Introduce Foreign Method** اصلاح کن:

csharp

```
public class InvoicePrinter
{
    public void PrintInvoice(DateTime dueDate)
    {
        // این منطق چند جا تکرار شده
        bool isOverdue = dueDate < DateTime.Now && dueDate.DayOfWeek != DayOfWeek.Saturday
            && dueDate.DayOfWeek != DayOfWeek.Sunday;

        if (isOverdue)
            Console.WriteLine("این فاکتور overdue هست!");
    }
}
```

راهنمایی: یه Foreign Method به اسم IsOverdue بساز که DateTime رو بگیره و چک کنه.

بفرست تا بررسی کنیم! 

خودت حلش کن

کد اصلاح شده با Introduce Foreign Method 

csharp

```

        public class InvoicePrinter
        {
            public void PrintInvoice(DateTime dueDate)
            {
                if (IsOverdue(dueDate))
                {
                    Console.WriteLine("این فاکتور overdue هست!");
                }
            }
        }

        Foreign Method - should be in DateTimeExtensions if possible //
        private static bool IsOverdue(DateTime date)
        {
            bool isWeekend = date.DayOfWeek == DayOfWeek.Saturday
                || date.DayOfWeek == DayOfWeek.Sunday;
            return date < DateTime.Now && !isWeekend;
        }
    }

```

مقایسه

نسخه اصلاح	کد اصلی	
✓ IsOverdue(dueDate)	شرط طولانی و نامفهوم	خوانایی
✓ IsOverdue هر جا قابل استفاده ست	نیست	استفاده مجدد
✓ فقط به متد	باید همه جا رو بگردی	تغییر منطق

نکته: در #C بهتره از Extension Method استفاده کنی

csharp

```

public static class DateTimeExtensions
{
    public static bool IsOverdue(this DateTime date)
    {
    }
}

```

```
bool isWeekend = date.DayOfWeek == DayOfWeek.Saturday
;date.DayOfWeek == DayOfWeek.Sunday ||
;return date < DateTime.Now && !isWeekend
{
{

// استفاده:
if (dueDate.IsOverdue())
;Console.WriteLine("Overdue!")
```

Introduce Foreign Method تموم شد!

بریم سراغ **Introduce Local Extension**؟ (آخرین تکنیک!) متنش رو بفرست! 